
Saudi Location Intelligence MCP Server

Technical Documentation

*Complete codebase walkthrough with architecture diagrams,
data flow charts, and module-level explanations*

Version 1.0.0

April 18, 2026

Contents

1	Project Overview	5
1.1	What Is This Project?	5
1.2	Key Capabilities	5
1.3	Technology Stack	6
1.4	High-Level Architecture	6
2	Configuration Layer	7
2.1	config.json — Runtime Configuration File	7
2.2	config.py — Configuration Module	7
2.2.1	Responsibility	7
2.2.2	Loading Flow	8
2.2.3	Exported Constants	9
2.2.4	Backward-Compatible Wrappers	9
3	Logging	10
3.1	logging_config.py — Structured Logging	10
3.1.1	Responsibility	10
3.1.2	Key Functions	10
4	Data Models	11
4.1	models.py — Core Pydantic Models	11
4.1.1	DataHandle	11
4.1.2	SessionInfo	11
4.2	all_types/request_dtypes.py — API Request Models	12
5	Application Context	13
5.1	context.py — Dependency Injection Helper	13
5.1.1	Problem Solved	13
5.1.2	Solution	13
6	Core Modules	14

6.1	<code>core/session_manager.py</code> — Session Lifecycle	14
6.1.1	Responsibility	14
6.1.2	Storage Layout	14
6.1.3	Session Lifecycle Diagram	15
6.1.4	Key Methods	16
6.2	<code>core/handle_manager.py</code> — Data Storage	16
6.2.1	Responsibility	16
6.2.2	Handle Format	16
6.2.3	Data Flow	17
6.2.4	Cleanup Methods	17
6.3	<code>core/cleanup.py</code> — Background Cleanup Task	17
7	Utility Modules	19
7.1	<code>utils/json_handler.py</code> — Async JSON I/O	19
7.2	<code>utils/http_client.py</code> — Backend HTTP Client	19
7.3	<code>utils/auth_helpers.py</code> — Centralised Auth Check	21
8	MCP Server Entry Point	22
8.1	<code>mcp_server.py</code> — Server Bootstrap	22
8.1.1	Startup Sequence	22
8.1.2	FastMCPWithCORS	22
8.1.3	MCP Resources	22
8.1.4	Expected Terminal Output	23
9	Tool Modules	24
9.1	Generic Tool Execution Flow	24
9.2	<code>tools/auth_tools.py</code> — Authentication Tools	24
9.2.1	Authentication Flow	25
9.2.2	Registered Tools	25
9.3	<code>tools/geospatial.py</code> — Geospatial Data Fetcher	25
9.3.1	Tool: <code>fetch_geospatial_data</code>	25
9.4	<code>tools/optimize_sales_territories.py</code> — Territory Optimisation . .	27
9.4.1	Tool: <code>optimize_sales_territories</code>	27
10	Analysis Tools	29
10.1	<code>tools/analysis_tools/hub_analyzer.py</code> — Hub Expansion Analysis	29
10.1.1	Tool: <code>hub_expansion_analyzer</code>	29
10.2	<code>tools/analysis_tools/pharmacy_analyzer.py</code> — Pharmacy Site Se- lection	31
10.2.1	Tool: <code>generate_pharmacy_report</code>	31

11 Report Tools	34
11.1 tools/report_tools/generate_report.py — Report Generation . .	34
11.1.1 Tool: generate_territory_report	34
11.2 tools/report_tools/report_analysis.py — LLM Report Analysis .	35
11.2.1 Tool: report_analysis	35
12 End-to-End Data Flows	36
12.1 Complete Geospatial Analysis Workflow	36
12.2 Token Refresh Cycle	36
13 Deployment	37
13.1 Running Locally	37
13.2 Running with Docker	37
13.2.1 Dockerfile Highlights	37
13.3 Environment Variables Reference	38
14 Project File Index	39

List of Figures

1.1	High-level system architecture	6
2.1	Configuration loading sequence	8
6.1	Session creation, loading, and expiry sequence	15
6.2	HandleManager store and retrieve flow	17
6.3	Background cleanup loop	18
7.1	HTTP client request envelope and error handling	20
9.1	Generic tool execution sequence	24
9.2	user_login authentication flow	25
9.3	fetch_geospatial_data flow	26
10.1	hub_expansion_analyzer flow with optional report generation	30
10.2	generate_pharmacy_report flow	32
11.1	Report generation and LLM analysis flow	34

Chapter 1

Project Overview

1.1 What Is This Project?

The **Saudi Location Intelligence MCP Server** is a server built on the [Model Context Protocol \(MCP\)](#) that exposes geospatial analysis tools to AI clients (such as Claude) or developer tools such as the MCP Inspector.

It acts as a *bridge* between an AI model and a backend REST API that holds Saudi Arabia location data — points of interest, real estate, demographics, and business analytics. The AI client calls a *tool* (e.g. `fetch_geospatial_data`), the MCP server authenticates the request, proxies it to the backend, stores the result on disk, and returns a lightweight *handle* plus a human-readable summary back to the AI.

1.2 Key Capabilities

- **Geospatial data fetching** — POI, real estate, demographics for Saudi cities
- **Territory optimisation** — spatial clustering for sales-team planning
- **Hub expansion analysis** — multi-criteria site scoring with AI reports
- **Pharmacy site selection** — weighted scoring across traffic, competition, demographics
- **Academic & executive report generation** — markdown reports with statistical analysis
- **LLM-powered report analysis** — Gemini-backed natural language Q&A over reports
- **Session management** — JWT authentication with automatic token refresh

1.3 Technology Stack

Layer	Technology
Protocol	Model Context Protocol (MCP) via <code>mcp[cli]</code>
Web framework	Starlette / uvicorn (SSE transport)
Data validation	Pydantic v2
HTTP client	aiohttp (async)
File I/O	aiofiles + custom file-locking
AI analysis	LangChain + Google Gemini
Runtime	Python 3.13
Container	Docker (python:3.13-slim + uv)
Build	tectonic (this document)

1.4 High-Level Architecture

Figure 1.1 shows all major components and how they connect.

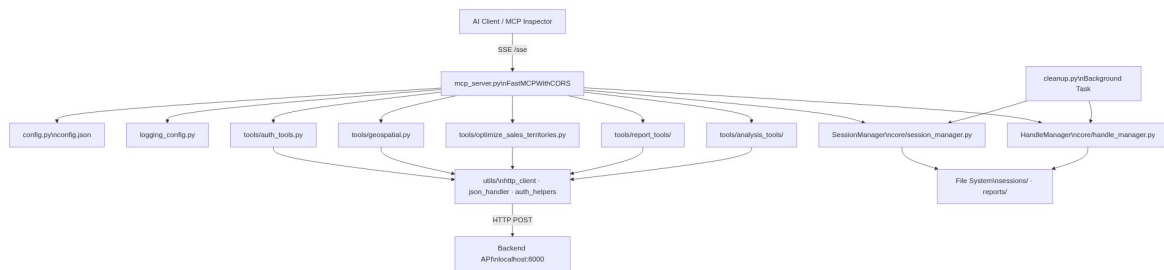


Figure 1.1: High-level system architecture

The server starts as a single Python process. An AI client connects over **SSE** (Server-Sent Events) to `/sse`. The MCP layer routes each tool call to the registered handler, which uses shared `SessionManager` and `HandleManager` singletons, calls the backend over HTTP, and writes results to disk.

Chapter 2

Configuration Layer

2.1 config.json — Runtime Configuration File

File: config.json

Format: JSON

config.json lives at the project root and provides defaults that can be overridden at runtime by environment variables (important for Docker deployments):

```
1 {  
2   "backend_url":      "http://localhost:8000",  
3   "server_host":     "0.0.0.0",  
4   "server_port":     8001,  
5   "cors_origins":    "*",  
6   "session_ttl_hours": 8,  
7   "cleanup_interval_hours": 1,  
8   "sessions_path":   "sessions",  
9   "reports_path":    "reports"  
10 }
```

Listing 2.1: Typical config.json

2.2 config.py — Configuration Module

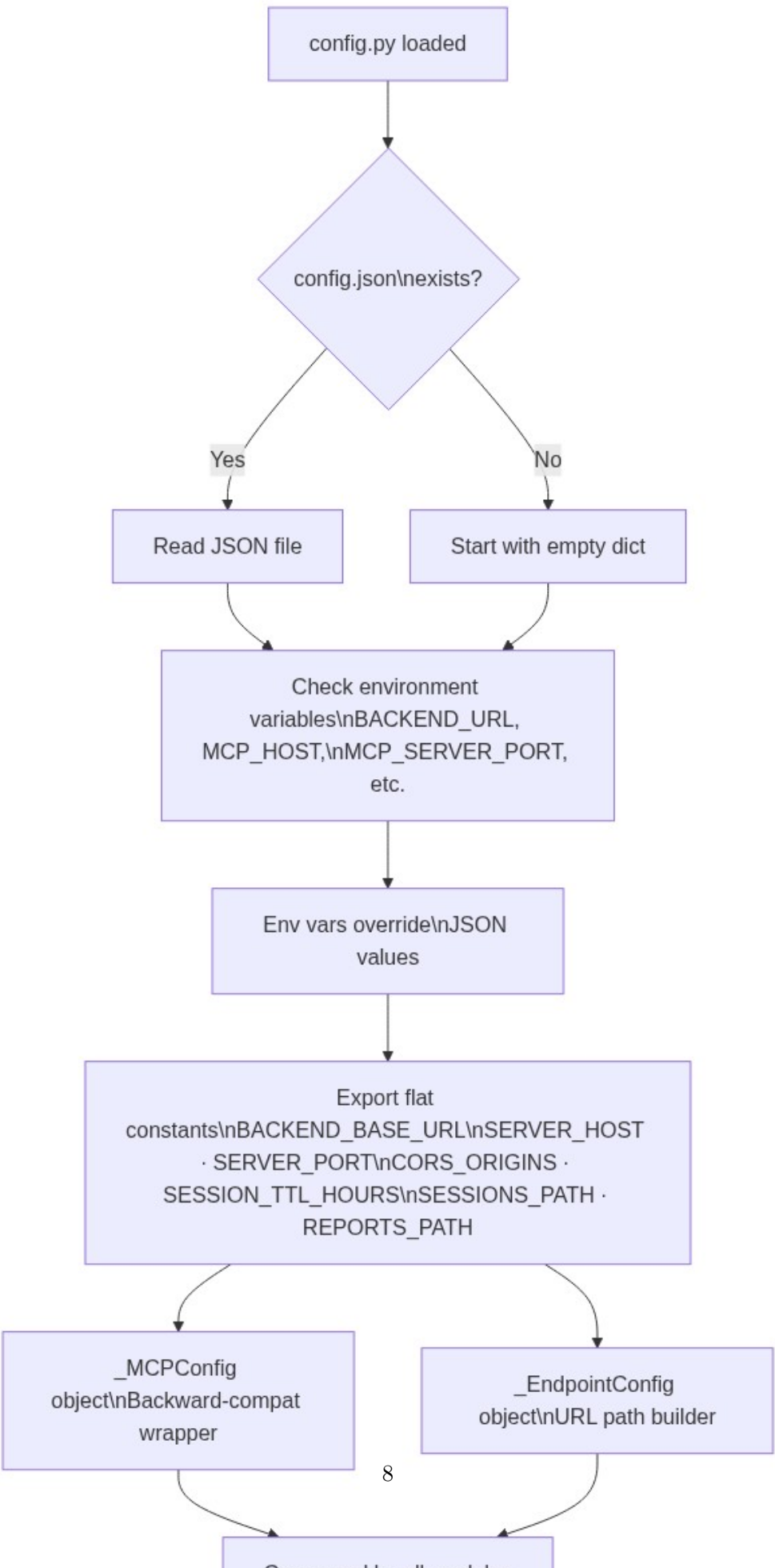
File: config.py

Imports used by: every module

2.2.1 Responsibility

config.py is the *single source of truth* for all configuration. It loads config.json once at import time, then checks environment variables and lets them override the JSON values. Every other module imports constants from here — no module reads environment variables directly.

2.2.2 Loading Flow



2.2.3 Exported Constants

Constant	Env Override	Purpose
BACKEND_BASE_URL	BACKEND_URL	URL of the backend REST API
SERVER_HOST	MCP_HOST	Bind address for uvicorn
SERVER_PORT	MCP_SERVER_PORT	Port for uvicorn
CORS_ORIGINS	CORS_ORIGINS	Comma-separated allowed origins
SESSION_TTL_HOURS	SESSION_TTL_HOURS	Session expiry window
CLEANUP_INTERVAL_HOURS	CLEANUP_INTERVAL_HOURS	Background cleanup cadence
SESSIONS_PATH	—	Absolute path to sessions directory
REPORTS_PATH	—	Absolute path to reports directory

2.2.4 Backward-Compatible Wrappers

Two small classes wrap the flat constants so that existing call-sites that accessed `config.session_ttl_hours` or `ENDPOINTS.login` continue to work without changes:

- `_MCPConfig` (config instance) — exposes `session_ttl_hours`, `cleanup_interval_hours`, `temp_storage_path`
- `_EndpointConfig` (ENDPOINTS instance) — builds full endpoint paths by prepending `/fastapi/` to route names (`.login`, `.fetch_dataset`, etc.)

Chapter 3

Logging

3.1 logging_config.py — Structured Logging

File: logging_config.py

3.1.1 Responsibility

Provides a consistent, structured logging setup with two output channels:

- **stderr** — always visible in the terminal / Docker logs
- **Log files** — one file per session stored in `sessions/<id>/session.log`

3.1.2 Key Functions

Function	Description
<code>setup_main_logging()</code>	Called once at startup; attaches stderr + file handler to the root logger
<code>get_logger(name)</code>	Returns a <code>logging.Logger</code> ; ensures main logging is initialised first
<code>setup_session_logging(session_id)</code>	Creates a session-scoped file handler; all tool log output goes here
<code>end_session_logging(session_id)</code>	Flushes and removes the session handler when the session ends

Note: Every module calls `get_logger(__name__)` at module level, so log messages always carry the originating module's name for easy filtering.

Chapter 4

Data Models

4.1 `models.py` — Core Pydantic Models

File: `models.py`

Two Pydantic models represent runtime state persisted to disk as JSON:

4.1.1 `DataHandle`

Metadata for a single stored data file. Fields:

Field	Type	Meaning
<code>session_id</code>	<code>str</code>	Which session owns this data
<code>data_type</code>	<code>str</code>	Category: <code>geospatial_data</code> , <code>hub_analysis</code> , etc.
<code>location</code>	<code>str</code>	City or region slug
<code>expires_at</code>	<code>datetime</code>	When this handle expires
<code>file_path</code>	<code>str</code>	Absolute path to the JSON file on disk
<code>summary</code>	<code>dict</code>	Human-readable summary metadata
<code>schema</code>	<code>dict</code>	Optional schema description

4.1.2 `SessionInfo`

Represents a user session. Fields:

Field	Type	Meaning
<code>session_id</code>	<code>str</code>	UUID for this session
<code>created_at</code>	<code>datetime</code>	Creation timestamp
<code>expires_at</code>	<code>datetime</code>	Expiry timestamp
<code>data_handles</code>	<code>list[str]</code>	Filenames of stored data blobs
<code>user_id</code>	<code>str</code> <code>None</code>	Authenticated user ID
<code>id_token</code>	<code>str</code> <code>None</code>	JWT ID token
<code>refresh_token</code>	<code>str</code> <code>None</code>	JWT refresh token
<code>token_expires_at</code>	<code>datetime</code> <code>None</code>	When the ID token expires

4.2 `all_types/request_dtypes.py` — API Request Models

File: `all_types/request_dtypes.py`

Pydantic models that mirror the backend API's expected request bodies. They are built by the tool functions and serialised with `.model_dump()` before being sent over HTTP.

Model	Purpose
<code>Coordinate</code>	lat/lng pair
<code>ReqCityCountry</code>	city_name + country_name
<code>UserId</code>	user_id string
<code>BooleanQuery</code>	boolean_query search string
<code>ReqFetchDataset</code>	Full geospatial fetch request (inherits <code>Coordinate</code> + <code>ReqCityCountry</code>)
<code>ReqClustersForSalesManData</code>	Territory clustering request (inherits <code>BooleanQuery</code> + <code>UserId</code> + <code>ReqCityCountry</code>)

Chapter 5

Application Context

5.1 context.py — Dependency Injection Helper

File: context.py

5.1.1 Problem Solved

Tool functions are defined inside `register_*(mcp)` functions. They need access to the global `session_manager` and `handle_manager` singletons created in `mcp_server.py`. Importing them directly would create circular imports.

5.1.2 Solution

`context.py` defines `AppContext` (a `@dataclass`) and a helper `get_app_context(mcp)` that imports the singletons *lazily* at call time:

```
1 def get_app_context(mcp: "FastMCP") -> AppContext:
2     from mcp_server import session_manager, handle_manager
3     return AppContext(
4         session_manager=session_manager,
5         handle_manager=handle_manager,
6     )
```

Listing 5.1: `get_app_context` helper

Every tool calls `get_app_context(mcp)` at the top of its body, giving it fully typed access to both managers without any import-time circular dependency.

Chapter 6

Core Modules

6.1 core/session_manager.py — Session Lifecycle

File: core/session_manager.py Class: SessionManager

6.1.1 Responsibility

Manages the full lifecycle of a user session: creation, persistence to disk, authentication token storage, token refresh, and expiry.

6.1.2 Storage Layout

```
1 sessions/  
2   <session_id>/  
3     session_metadata.json    <- SessionInfo serialised to JSON  
4     session.log              <- Session-scoped log file  
5     <data_handle>.json       <- Stored data blobs (managed by  
                               HandleManager)
```

Listing 6.1: Session directory structure on disk

6.1.3 Session Lifecycle Diagram

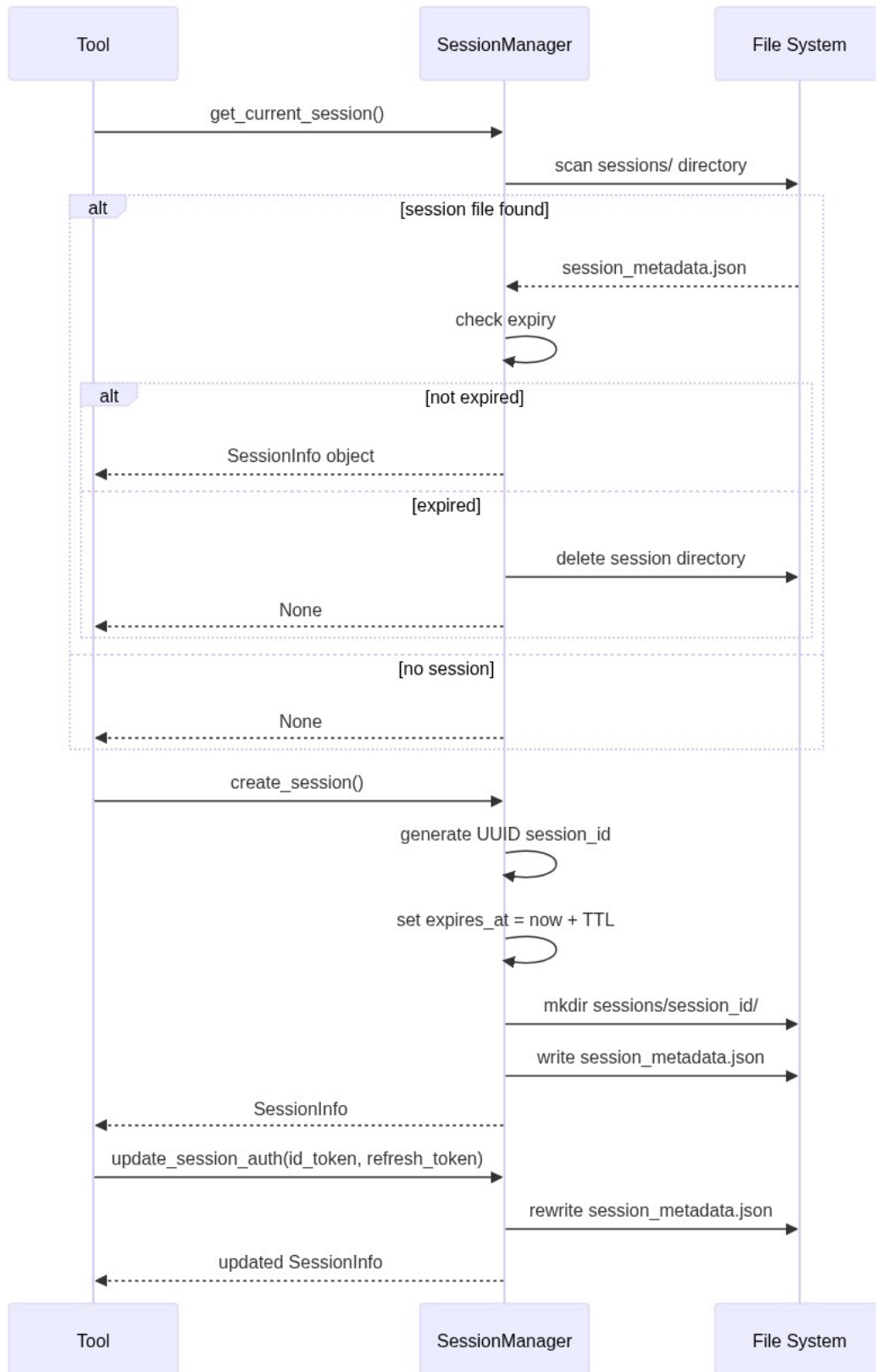


Figure 6.1: Session creation, loading, and expiry sequence

6.1.4 Key Methods

Method	Description
<code>create_session()</code>	Generates UUID, sets expiry from <code>SESSION_TTL_HOURS</code> , writes metadata to disk
<code>get_current_session()</code>	Scans disk for existing non-expired session; returns <code>None</code> if none found
<code>load_session(path)</code>	Reads <code>session_metadata.json</code> ; deletes directory if expired
<code>update_session_auth(tokens)</code>	Writes JWT tokens + expiry back to the metadata file
<code>get_valid_id_token()</code>	Returns current token if valid; otherwise calls the refresh endpoint automatically
<code>cleanup_session(session_id)</code>	Removes session directory from disk

6.2 `core/handle_manager.py` — Data Storage

File: `core/handle_manager.py` **Class:** `HandleManager`

6.2.1 Responsibility

Provides a simple *handle-based* key-value store backed by JSON files. A handle is just a filename — tools store large API responses and get back a compact handle string they can pass to other tools.

6.2.2 Handle Format

```

1 {data_type}_{location}_{YYYYMMDDHHMMSSFF}_{uuid8}.json
2 # Example:
3 geospatial_data_riyadh_20260418_213401_123456_a1b2c3d4.json

```

Listing 6.2: Handle filename format

The microsecond component (`%f`) ensures uniqueness even for back-to-back calls within the same second.

6.2.3 Data Flow

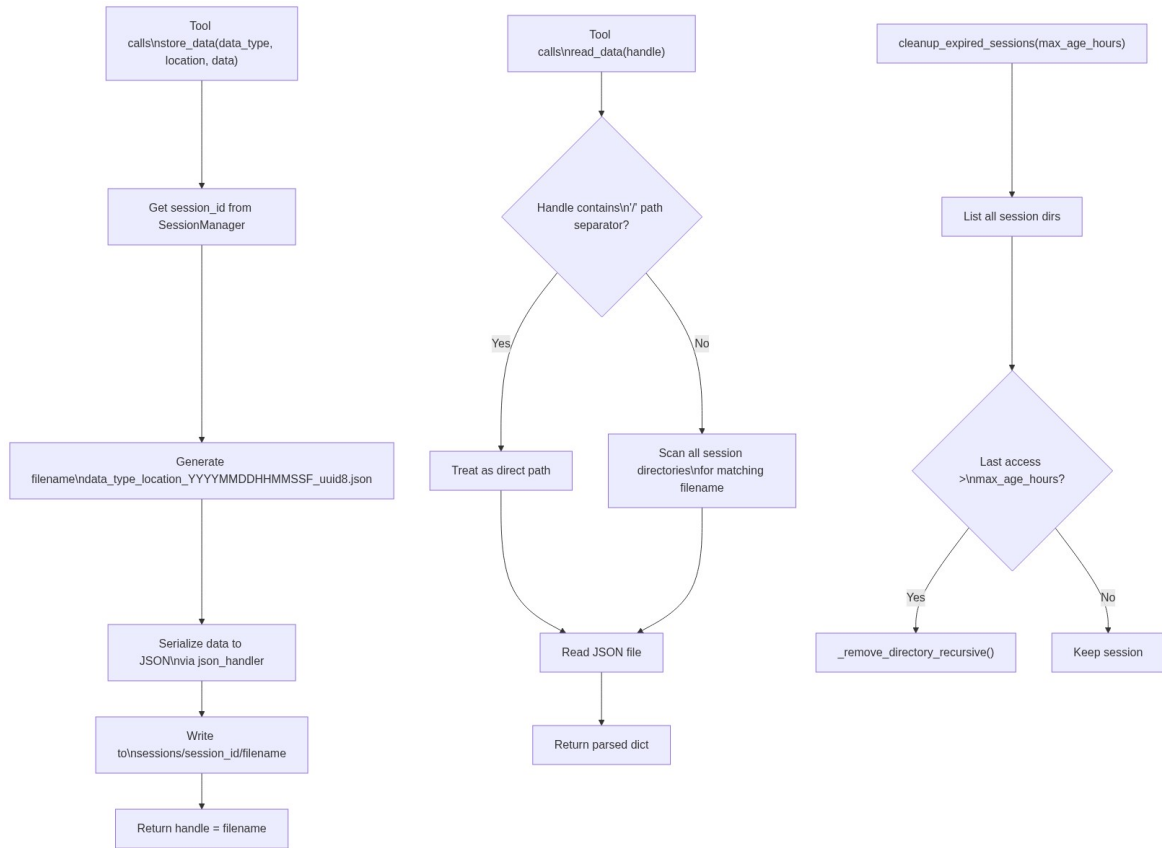


Figure 6.2: HandleManager store and retrieve flow

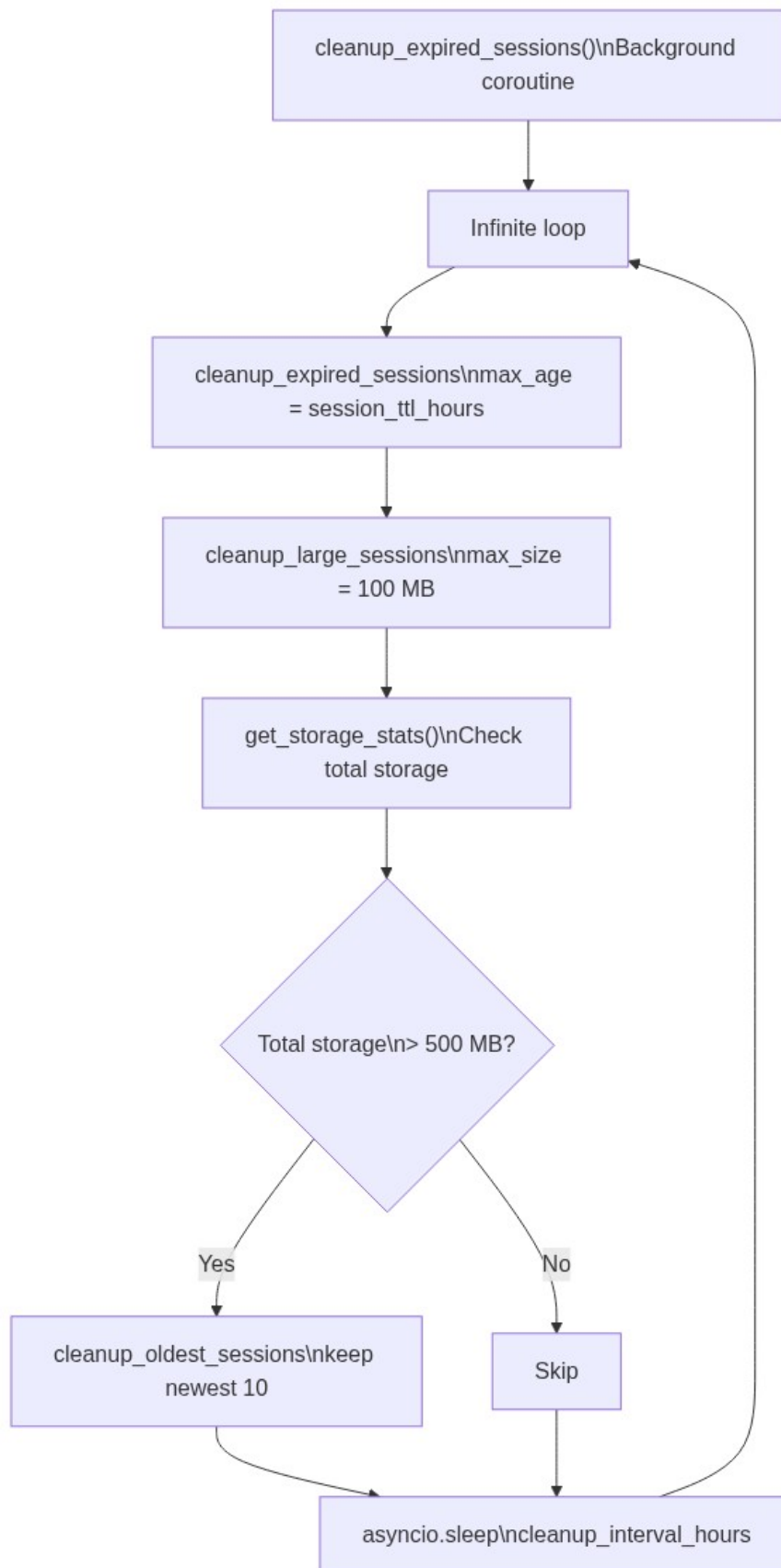
6.2.4 Cleanup Methods

Method	Trigger Condition
<code>cleanup_expired_sessions(max_age_hours)</code>	Sessions older than TTL
<code>cleanup_large_sessions(max_size_mb)</code>	Sessions exceeding 100 MB
<code>cleanup_oldest_sessions(keep_n)</code>	Total storage > 500 MB
<code>get_storage_stats()</code>	Returns per-session and total size metrics

6.3 core/cleanup.py — Background Cleanup Task

File: `core/cleanup.py`

An infinite `async` coroutine that runs in the background throughout the server's lifetime. It enforces three storage policies in sequence on every tick:

**Figure 6.3:** Background cleanup loop

Chapter 7

Utility Modules

7.1 `utils/json_handler.py` — Async JSON I/O

File: `utils/json_handler.py`

Handles reading and writing JSON files safely in an async context:

- **FileLock** — simple async lock that prevents two coroutines from writing the same file simultaneously (avoids corruption)
- **to_serializable()** — recursively converts Pydantic models, `datetime` objects, sets, etc. to plain Python types before JSON serialisation
- **use_json(path, mode, data)** — async read (`'r'`) or write (`'w'`) with automatic file locking; used by both managers

7.2 `utils/http_client.py` — Backend HTTP Client

File: `utils/http_client.py`

All communication with the backend REST API goes through two functions here:

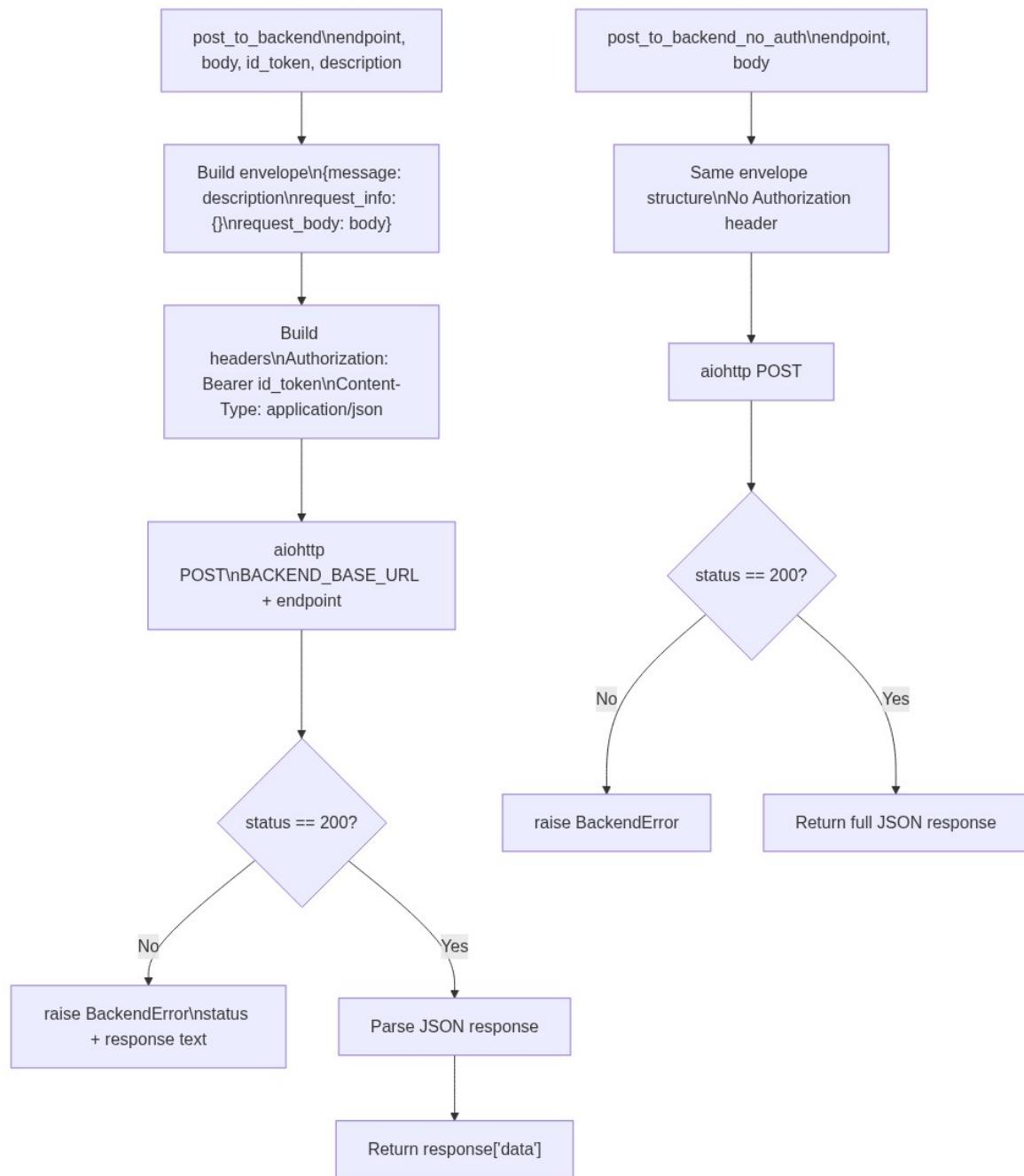


Figure 7.1: HTTP client request envelope and error handling

Every request wraps the payload in a standard envelope:

```

1 {
2     "message":      description,      # human-readable
3     "request_info": {},               # reserved for future
4     "request_body": body              # actual payload (dict from
5 }                                     Pydantic .model_dump())

```

Listing 7.1: Request envelope structure

BackendError is raised for any non-200 response and carries the HTTP status code and raw response text so tool functions can surface a helpful error message to the AI client.

7.3 `utils/auth_helpers.py` — Centralised Auth Check

File: `utils/auth_helpers.py`

`require_auth(session_manager)` is called at the top of *every* tool function that needs an authenticated user. It:

1. Calls `session_manager.get_current_session()`
2. If no session exists, creates a new one
3. Checks that the session has a valid `id_token`
4. Calls `get_valid_id_token()` which refreshes the token if expired
5. Returns `(session, user_id, id_token)` on success, or an error string

Returning a plain error string (instead of raising an exception) means the MCP tool can pass it back to the AI client as a readable message.

Chapter 8

MCP Server Entry Point

8.1 mcp_server.py — Server Bootstrap

File: `mcp_server.py` Entry point: `python mcp_server.py`

8.1.1 Startup Sequence

When you run `python mcp_server.py`, the following happens in order:

1. **Module imports** — all tools, core managers, and config loaded
2. **Global singletons created** — `SessionManager()` then `HandleManager(session_manager)`
3. **FastMCPWithCORS instantiated** — server named "saudi-location-intelligence"
4. **All tools registered** — seven `register_*(mcp)` calls
5. **main() called** — logs startup info, calls `mcp.run("sse")`
6. **Uvicorn started** — listens on `SERVER_HOST:SERVER_PORT`

8.1.2 FastMCPWithCORS

This subclass of `FastMCP` adds two overrides:

Override	Why
<code>sse_app()</code>	Wraps the Starlette app with <code>CORSMiddleware</code> so browser-based clients (MCP Inspector) can connect
<code>run()</code>	Forces the bind address to <code>SERVER_HOST</code> (default <code>0.0.0.0</code>) instead of <code>127.0.0.1</code> , which is required for Docker

8.1.3 MCP Resources

Two read-only *resources* are registered (queryable by MCP clients):

URI	Returns
<code>session://current</code>	Current session ID and expiry timestamp (or “No active session”)
<code>config://server</code>	Server configuration summary: TTL, storage path, available tools

8.1.4 Expected Terminal Output

```
1 INFO mcp_server - [wrench] Main logging initialized
2 INFO mcp_server - [SA] Saudi Location Intelligence MCP Server
3 INFO mcp_server - Starting SSE transport on
  http://0.0.0.0:8001/sse
4 INFO mcp_server - Connect MCP Inspector to:
  http://localhost:8001/sse
5 INFO uvicorn - Started server process [PID]
6 INFO uvicorn - Uvicorn running on http://0.0.0.0:8001
```

Listing 8.1: Terminal output on startup

Chapter 9

Tool Modules

9.1 Generic Tool Execution Flow

All tools follow the same pattern. Figure 9.1 shows the complete sequence from AI client call to response.

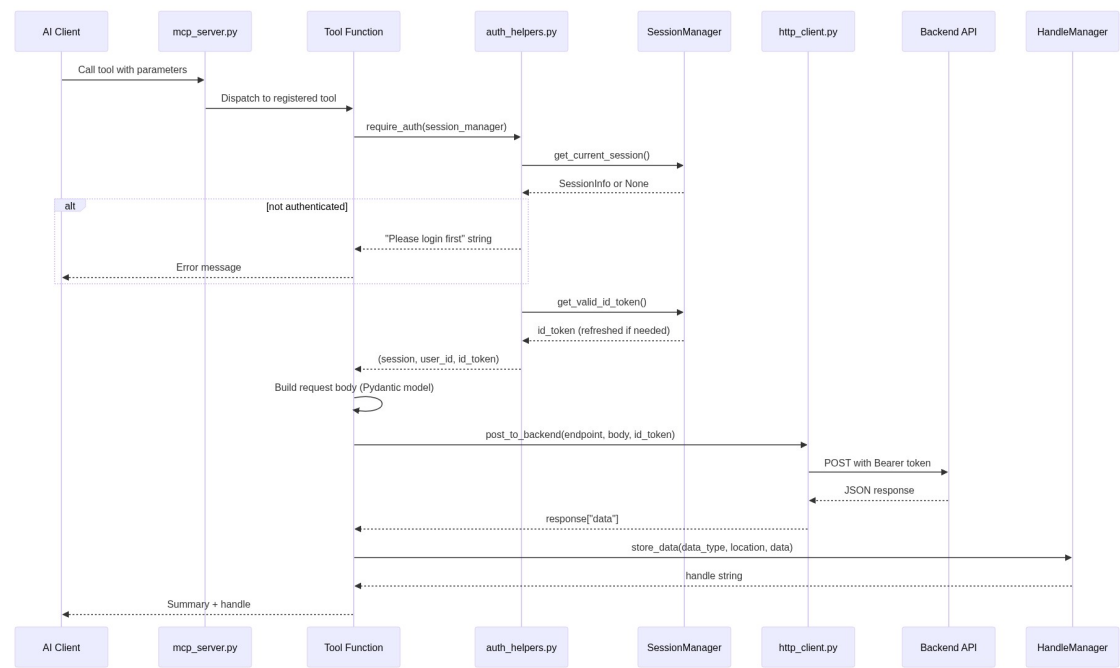


Figure 9.1: Generic tool execution sequence

9.2 tools/auth_tools.py — Authentication Tools

File: tools/auth_tools.py Registers: 4 tools

9.2.1 Authentication Flow

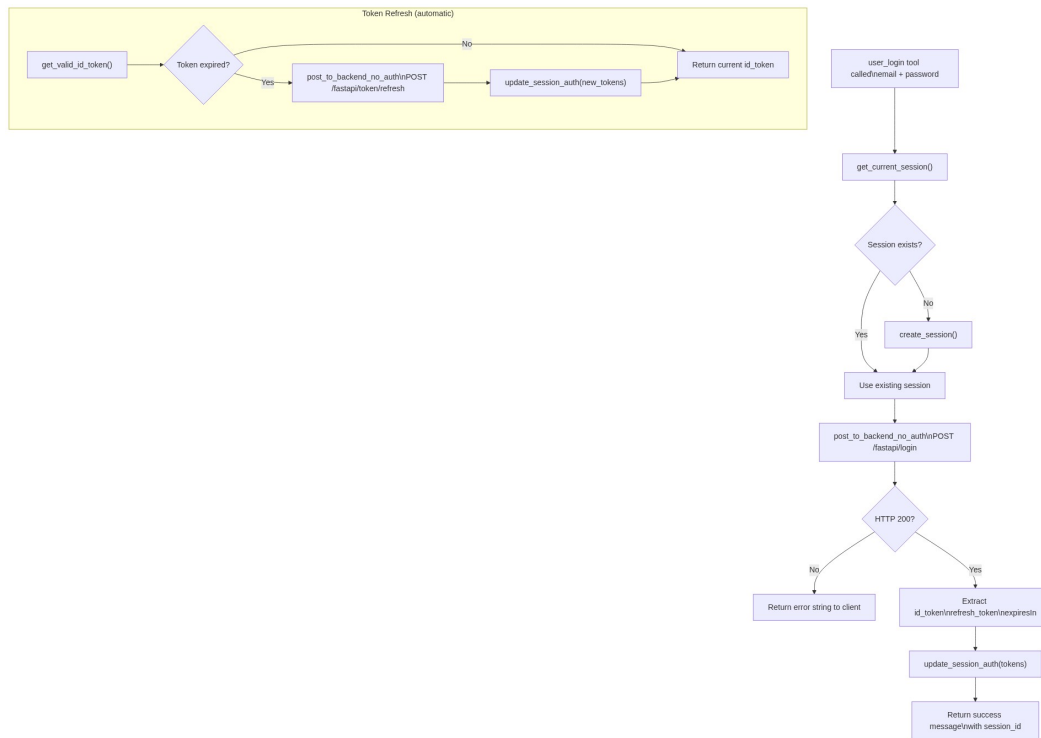


Figure 9.2: user_login authentication flow

9.2.2 Registered Tools

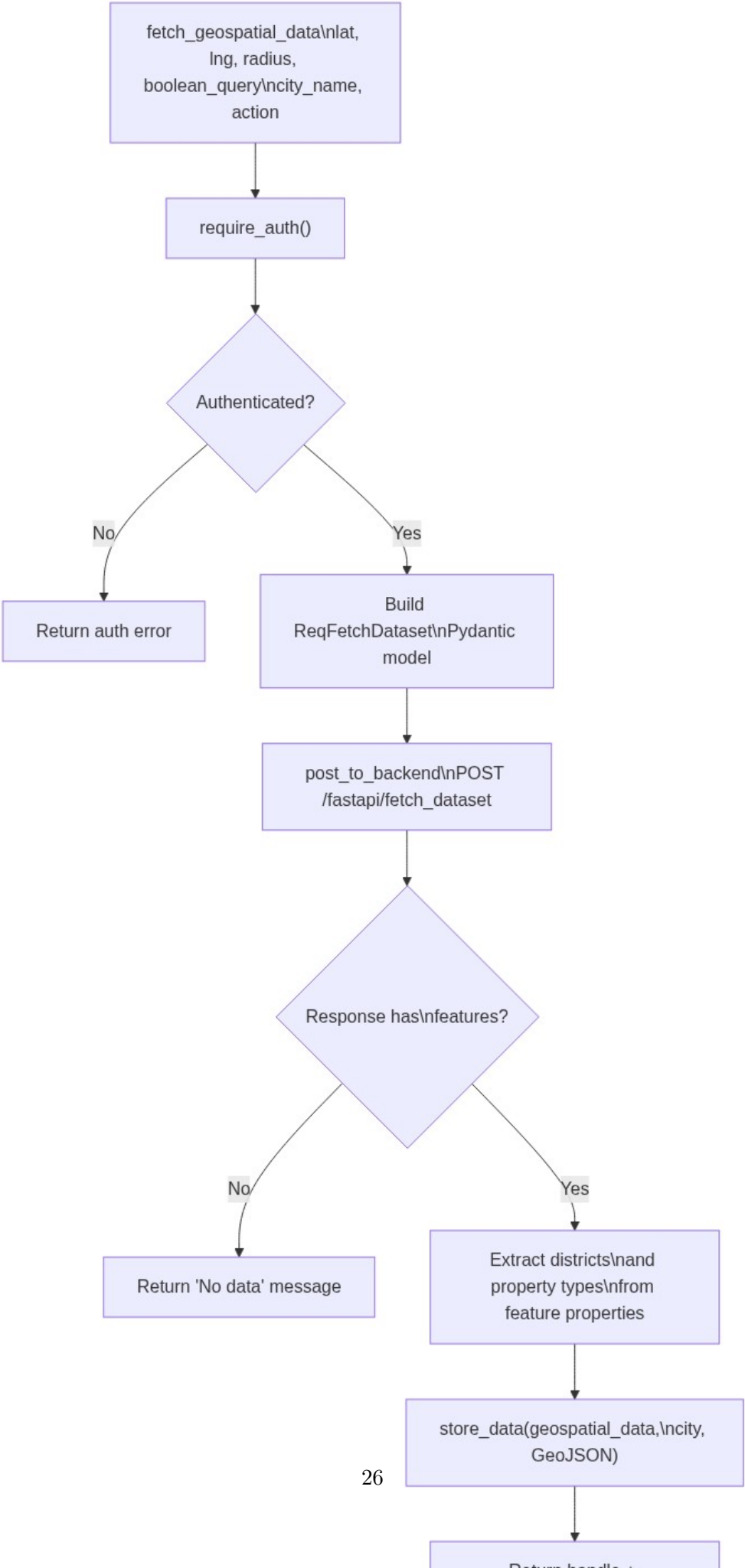
Tool name	Description
user_login	Authenticates with email + password; stores JWT tokens in session
list_stored_data	Lists all data files currently stored in the session directory
get_data_from_handle	Reads a stored JSON file by handle and returns its content
get_current_session_logs	Returns the last N lines of the session log file

9.3 tools/geospatial.py — Geospatial Data Fetcher

File: tools/geospatial.py **Registers:** 1 tool

9.3.1 Tool: fetch_geospatial_data

Fetches GeoJSON data from the backend for a given Saudi city and search query. The full dataset is stored server-side; only a handle + summary is returned.



Parameters

Parameter	Default	Description
lat, lng	required	Search centre coordinates
radius	5000 m	Search radius in metres
boolean_query	required	e.g. "restaurant AND NOT fast_food"
city_name	required	Saudi city (Riyadh, Jeddah, etc.)
action	"sample"	"sample" (20 records) or "full data"
include_rating_info	False	Include review/rating data
include_only_sub_properties	True	Return compact property set

Return Value

```
1 OK Data fetched. Handle:
   'geospatial_data_riyadh_20260418_214501_123456_a1b2c3d4.json'.
2 Summary: 47 records found for 'warehouse OR logistics' in
   Riyadh.
3 Districts: ['Al Malaz', 'Al Quds', 'Al Salam'].
4 Types: ['warehouse', 'logistics_center'].
```

Listing 9.1: Example return value

9.4 tools/optimize_sales_territories.py — Territory Optimisation

File: tools/optimize_sales_territories.py Registers: 1 tool

9.4.1 Tool: optimize_sales_territories

Uses spatial clustering on the backend to divide a city’s business locations into balanced sales territories. Metrics returned include:

- Market equity score (Coefficient of Variation across territories)
- Population distribution per territory
- Accessibility analysis (drive-time coverage)
- Service gap identification

Key Parameters

Parameter	Default	Description
city_name	required	Target Saudi city
num_sales_man	required	Number of sales representatives / territories
distance_limit	2.5 km	Maximum intra-territory distance
boolean_query	required	Business type to cluster
include_raw_data	False	Include raw clustering data in handle

Chapter 10

Analysis Tools

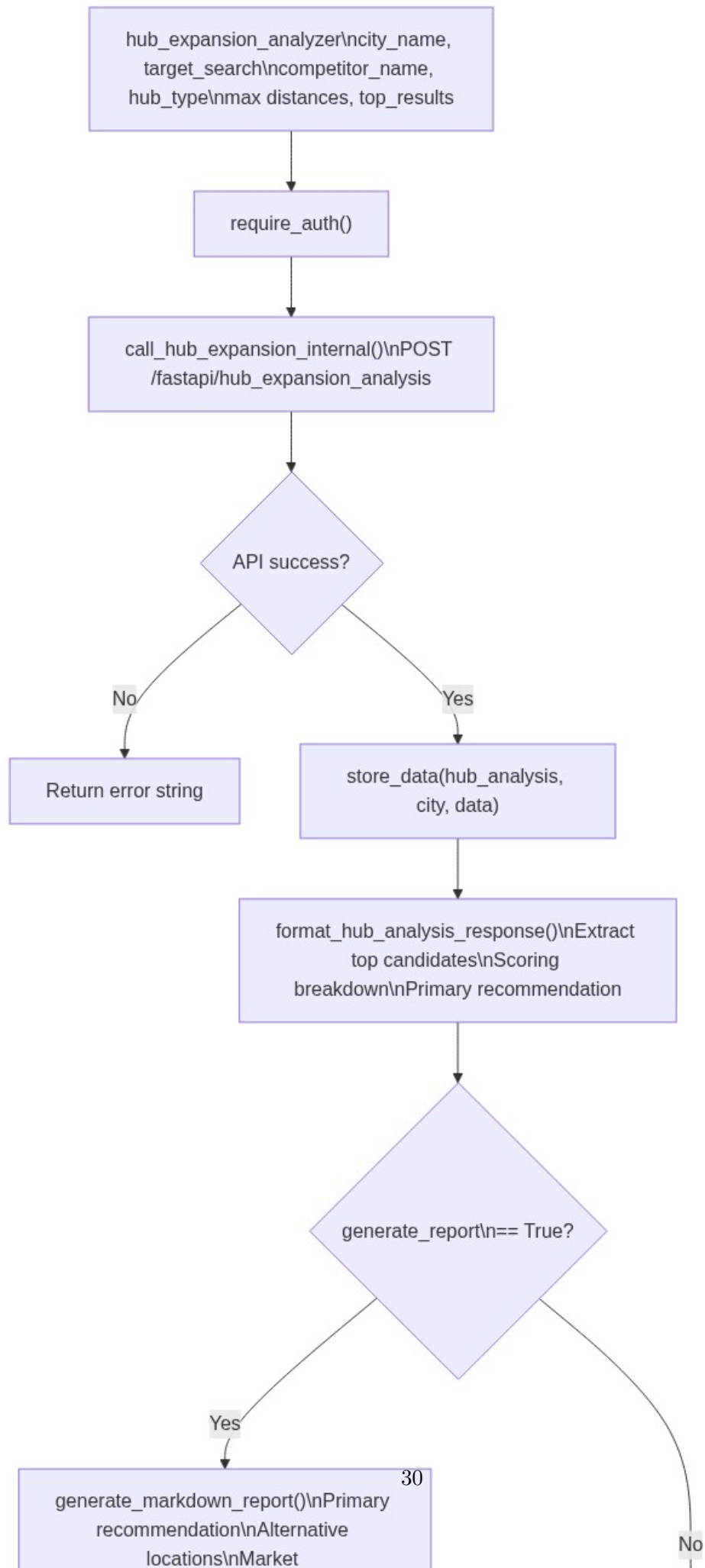
10.1 `tools/analysis_tools/hub_analyzer.py` — Hub Expansion Analysis

File: <code>tools/analysis_tools/hub_analyzer.py</code> Registers: 1 tool

10.1.1 Tool: `hub_expansion_analyzer`

Scores potential hub locations using five weighted criteria:

1. Target proximity — distance to key demand points
2. Population access — how many people can reach the hub
3. Rent efficiency — cost vs. footfall ratio
4. Competitive advantage — distance from competing hubs
5. Population coverage — broader catchment area



Report Generation

When `generate_report=True`, the tool creates a Markdown file in `reports/` containing:

- Primary recommendation with full scoring breakdown
- Alternative candidate locations
- Market analysis and competitor mapping
- Methodology explanation

10.2 `tools/analysis_tools/pharmacy_analyzer.py` — Pharmacy Site Selection

File: <code>tools/analysis_tools/pharmacy_analyzer.py</code> Registers: 1 tool
--

10.2.1 Tool: `generate_pharmacy_report`

Scores pharmacy locations using five configurable weight factors:

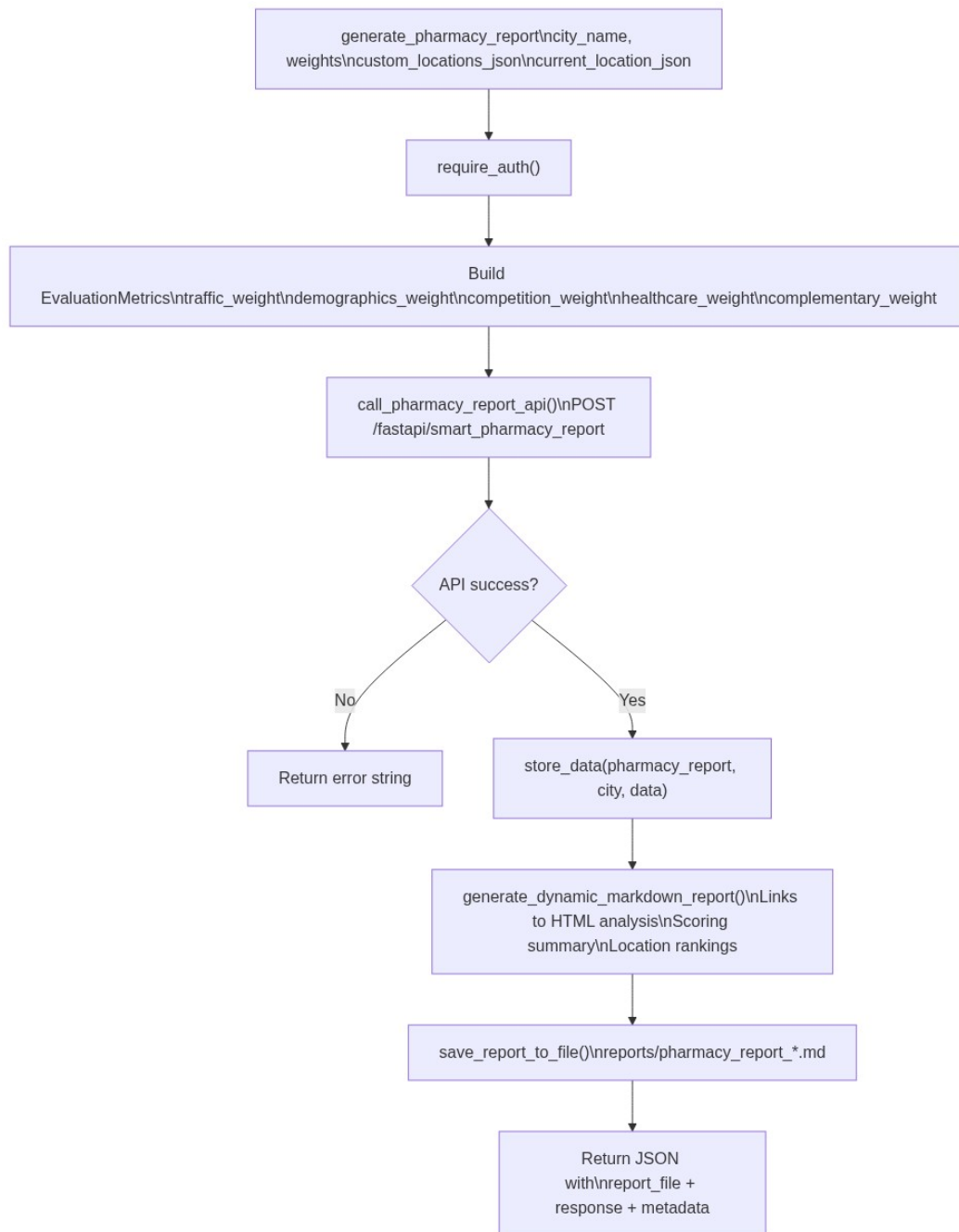


Figure 10.2: generate_pharmacy_report flow

Scoring Weights (all default to equal)

Weight Parameter	What it measures
<code>traffic_weight</code>	Foot traffic and vehicular accessibility
<code>demographics_weight</code>	Population density and age distribution
<code>competition_weight</code>	Proximity to competing pharmacies
<code>healthcare_weight</code>	Nearby hospitals, clinics, medical centres
<code>complementary_weight</code>	Co-located complementary businesses (supermarkets, etc.)

Custom locations can be passed as a JSON string (`custom_locations_json`) to evaluate specific addresses instead of the backend's suggestions.

Chapter 11

Report Tools

11.1 tools/report_tools/generate_report.py — Report Generation

File: tools/report_tools/generate_report.py Registers: 1 tool

11.1.1 Tool: generate_territory_report

Takes a `data_handle` (from `optimize_sales_territories`) and produces a formatted Markdown report. Three report types are available:

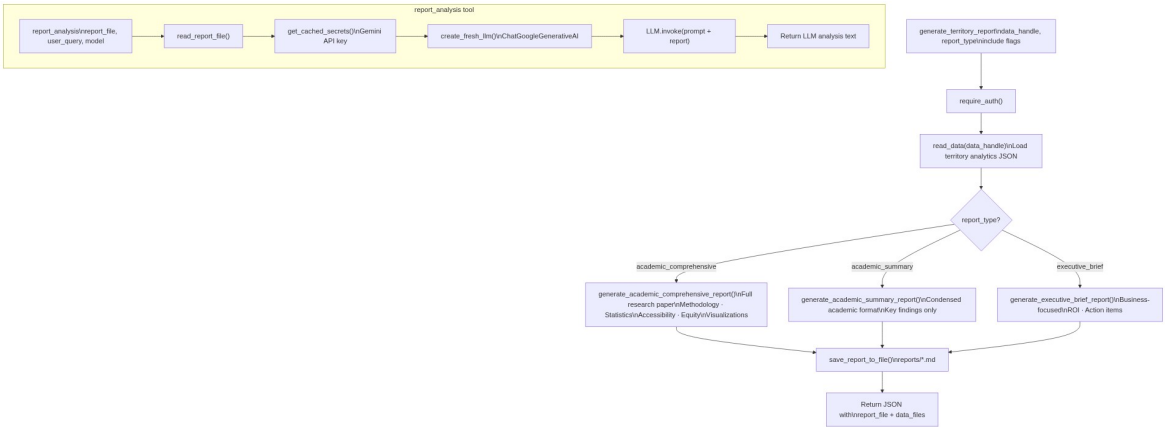


Figure 11.1: Report generation and LLM analysis flow

Report Type	Contents
academic_comprehensive	Full research paper: methodology, statistics, accessibility, equity analysis, bibliography
academic_summary	Condensed academic format; key findings and tables only
executive_brief	Business-focused: ROI, action items, risk summary

Statistical Analysis Functions

The report generator includes several helper functions that derive statistics from raw territory data:

- `extract_territory_metrics()` — mean, std, CV, min, max per metric
- `assess_balance_quality(cv)` — classifies balance as Excellent/Good/Moderate/Poor
- `generate_equity_analysis()` — validates whether territory sizes are balanced
- `generate_accessibility_analysis()` — service coverage within drive-time bands

11.2 tools/report_tools/report_analysis.py — LLM Report Analysis

File: tools/report_tools/report_analysis.py Registers: 1 tool

11.2.1 Tool: report_analysis

Reads a saved Markdown report from disk and sends it to a Google Gemini LLM along with the user's query. Returns a natural language answer.

Implementation Details

- Uses **LangChain** (`ChatGoogleGenerativeAI`) as the LLM abstraction
- Default model: `gemini-2.5-flash`
- API key loaded from `secrets/secrets_llm.json` (cached after first load)
- A fresh LLM instance is created per call to avoid event-loop conflicts
- Unicode characters in reports are sanitised before sending to avoid encoding issues

Chapter 12

End-to-End Data Flows

12.1 Complete Geospatial Analysis Workflow

A typical end-to-end usage from the AI client perspective:

1. **Login** — Call `user_login` with email + password. Session created, JWT stored.
2. **Fetch data** — Call `fetch_geospatial_data` with lat/lng and a boolean query. Receive a handle like `geospatial_data_riyadh_*.json`.
3. **Optimise territories** — Call `optimize_sales_territories` with `city_name`, `num_sales_man`, same query. Receive a territory analytics handle.
4. **Generate report** — Call `generate_territory_report` with the territory handle. Receive a Markdown report file path.
5. **Analyse report** — Call `report_analysis` with the report file path and a natural language question. Receive Gemini's answer.

12.2 Token Refresh Cycle

JWT ID tokens expire after `expiresIn` seconds (returned by the login endpoint). `get_valid_id_token()` checks expiry before every tool call and automatically calls `/fastapi/token/refresh` if needed. This is transparent to the AI client — it never needs to call `user_login` again within a session.

Chapter 13

Deployment

13.1 Running Locally

```
1 # Install dependencies
2 uv sync
3
4 # Start server
5 python mcp_server.py
6 # or
7 uv run slocator-mcp
8
9 # Connect MCP Inspector to:
10 # http://localhost:8001/sse
```

Listing 13.1: Local startup

13.2 Running with Docker

```
1 docker build -t slocator-mcp .
2
3 docker run -p 8001:8001 \
4     -e BACKEND_URL=http://your-backend:8000 \
5     -e MCP_SERVER_PORT=8001 \
6     -e SESSION_TTL_HOURS=8 \
7     slocator-mcp
```

Listing 13.2: Docker build and run

13.2.1 Dockerfile Highlights

- Base image: `python:3.13-slim`
- Package manager: `uv` (fast, no `pip`)
- Non-root user: `appuser` (UID 1000) for security
- Health check: HTTP GET to `/sse` endpoint on port 8001
- Runtime directories (`sessions/`, `reports/`, `logs/`) created at build time with correct ownership

13.3 Environment Variables Reference

Variable	Default	Description
BACKEND_URL	http://localhost:8000	Backend REST API base URL
MCP_HOST	0.0.0.0	Bind address
MCP_SERVER_PORT	8001	Port
CORS_ORIGINS	*	Comma-separated CORS origins
SESSION_TTL_HOURS	8	Session lifetime
CLEANUP_INTERVAL_HOURS	1	Cleanup loop interval

Chapter 14

Project File Index

File	Purpose
<code>mcp_server.py</code>	Entry point; server bootstrap and tool registration
<code>config.py</code>	Configuration loader with env-var overrides
<code>config.json</code>	Default runtime configuration
<code>logging_config.py</code>	Structured logging setup
<code>models.py</code>	<code>DataHandle</code> and <code>SessionInfo</code> Pydantic models
<code>context.py</code>	<code>AppContext</code> dataclass and lazy-import helper
<code>core/session_manager.py</code>	Session lifecycle, JWT storage, token refresh
<code>core/handle_manager.py</code>	File-backed key-value store; cleanup policies
<code>core/cleanup.py</code>	Infinite async cleanup coroutine
<code>utils/json_handler.py</code>	Async JSON file I/O with file locking
<code>utils/http_client.py</code>	Backend HTTP client; <code>BackendError</code>
<code>utils/auth_helpers.py</code>	<code>require_auth()</code> guard used by all tools
<code>all_types/request_dtypes.py</code>	Pydantic request models for backend API
<code>tools/auth_tools.py</code>	login, list data, read handle, session logs
<code>tools/geospatial.py</code>	<code>fetch_geospatial_data</code> tool
<code>tools/optimize_sales_territories.py</code>	<code>optimize_sales_territories</code> tool
<code>tools/analysis_tools/hub_analyzer.py</code>	<code>hub_expansion_analyzer</code> tool
<code>tools/analysis_tools/pharmacy_analyzer.py</code>	<code>generate_pharmacy_report</code> tool
<code>tools/report_tools/generate_report.py</code>	<code>generate_territory_report</code> tool
<code>tools/report_tools/report_analysis.py</code>	<code>report_analysis</code> LLM tool
<code>pyproject.toml</code>	Project metadata and dependencies
<code>Dockerfile</code>	Container build definition